Deep-Reinforcement learning Tasks

김영은

성공한 tasks

카트폴 스윙업

이중 역진자 스윙업

3~5중 역진자 밸런싱

공통적인 강화학습 계산 흐름

1. 현재상태 s_t 관측

2. 정책 $\pi_\theta(a|s)$로부터 행동 a_t 생성 (카트에 가하는 힘, 모터 움직임 등)

3. 물리 시뮬레이터가 미분방정식 적분하여 새로운 상태인 s_{t+1} 계산

4. 보상 계산 $r_t = r(s_t, a_t, s_{t+1})$

5. 여러 타임스텝을 모아 trajectory $\tau = (s_0, a_0, r_0, s_1, \ldots)$ 만듦

6. PPO로 θ 업데이트

Cart-pole Swing-up : state & action

$$s = \begin{bmatrix} x \\ \dot{x} \\ \theta \\ \dot{\theta} \end{bmatrix}$$

대표적 상태 벡터

x : 카트 위치

$\dot{x}$: 카트 속도

θ : 막대 각도 (보통 아래쪽이 0, 위쪽이 π 인 설정을 많이 씀)

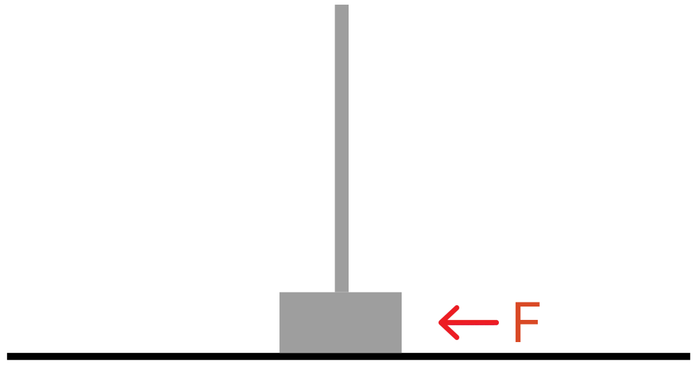
$\dot{\theta}$: 막대 각속도

Cart-pole Swing-up : state & action

행동 a 는 카트에 가하는 수평 힘 F 로 두는 경우가 많음

연속 행동이면 a∈R

이산이면 a∈ $\{-F_{\max}, +F_{\max}\}$



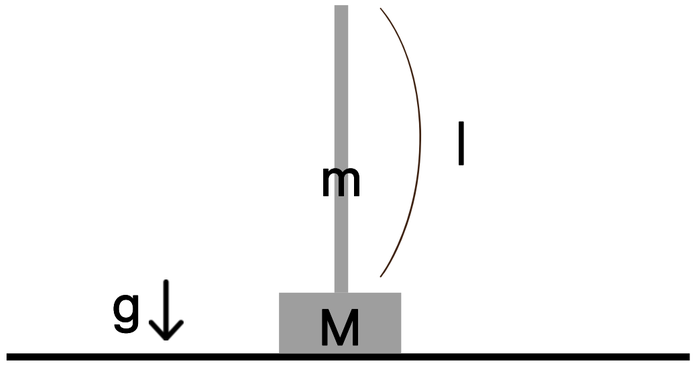
그럼 전체 변수들 (카트와 pole의 무제, 중력 등)을 어떻게 계산할 수 있을까?

Cart-pole Swing-up : 동역학 방정식

$$\ddot{\theta} = \frac{g\sin\theta + \cos\theta\left(\frac{-F - ml\dot{\theta}^2\sin\theta}{M+m}\right)}{l\left(\frac{4}{3} - \frac{m\cos^2\theta}{M+m}\right)}$$

$$\ddot{x} = \frac{F + ml(\dot{\theta}^2\sin\theta - \ddot{\theta}\cos\theta)}{M+m}$$

질량 M (카트), m (막대), 길이l (막대 중심까지), 중력 g 라고 할 때,
카트폴의 연속시간 동역학



하지만 Cart-pole은 연속적인 시간 환경에서 흘러가는데, 시간은 어떻게 계산하는가?

Cart-pole Swing-up : 동역학 방정식(real simulation)

실제 시뮬레이션은 이 미분방정식을 작은 시간 간격 Δt 로 적분

$$\dot{x}_{t+1} = \dot{x}_t + \ddot{x}_t \Delta t, \quad x_{t+1} = x_t + \dot{x}_{t+1} \Delta t$$

$$\dot{\theta}_{t+1} = \dot{\theta}_t + \ddot{\theta}_t \Delta t, \quad \theta_{t+1} = \theta_t + \dot{\theta}_{t+1} \Delta t$$

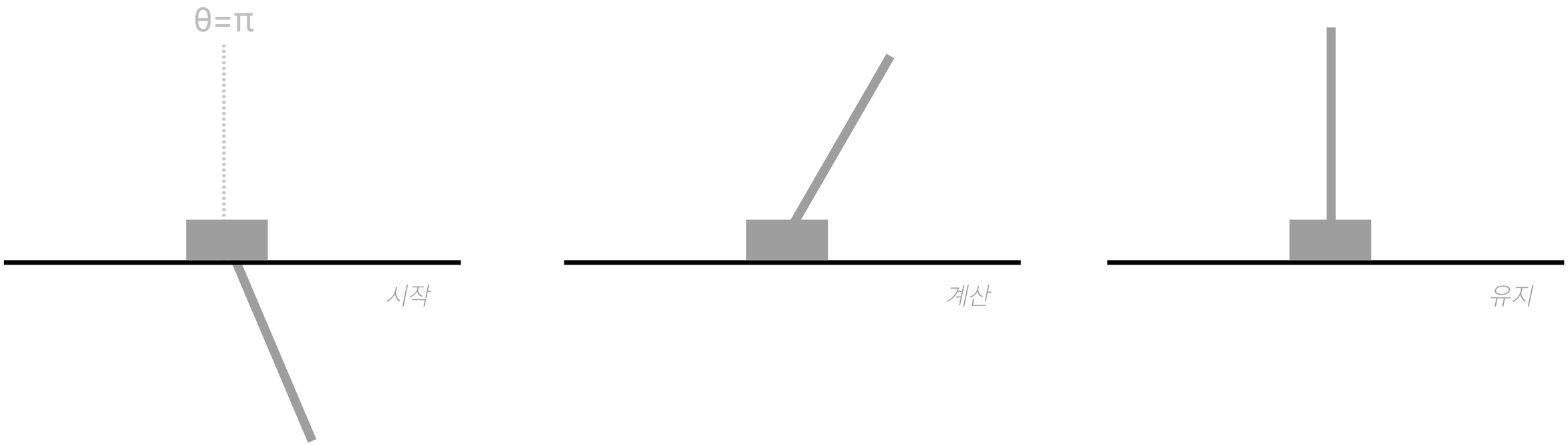
explicit Euler 사용시 계산식

새로운 상태 s_{t+1}

변수와 시간 계산 방법을 알았다. 그러면 강화학습에서 중요한 변수인 reward는 어떻게 계산하는가?

Cart-pole Swing-up : reward

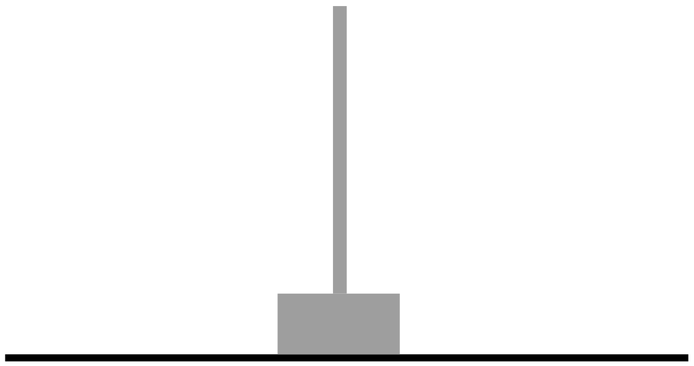
스윙업의 목표는 θ=π (위) 근처에 도달하고 유지하는 것



reward를 계산하는 방법에는 무엇이 있는가?

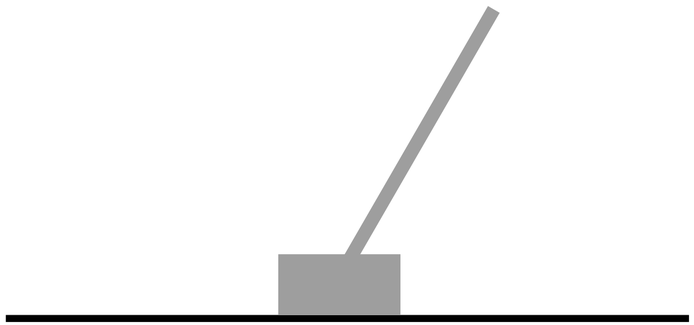
Cart-pole Swing-up : reward (각도 기반)

$$r_t = -\alpha(\theta_t - \pi)^2 - \beta\dot{\theta}_t^2 - \gamma x_t^2 - \delta\dot{x}_t^2$$

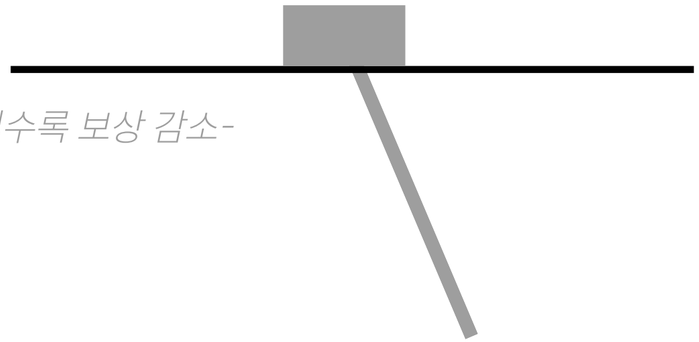


목표 상태

$$\theta_{\text{target}} = \pi,\ \dot{\theta} = 0$$



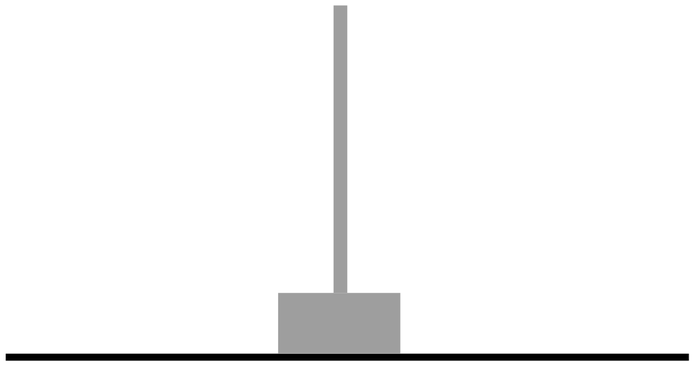
-목표 상태에서 멀어질수록 보상 감소-



각도 기반 reward 말고 다른 방법도 있는가?

Cart-pole Swing-up : reward (에너지 기반)

막대의 전체 E : $E(\theta, \dot{\theta}) = mgl(1 - \cos\theta) + \frac{1}{2}ml^2\dot{\theta}^2$



위쪽 평형점의 에너지

$E_{\text{up}} = 2mgl$

보상 설정

$$r_t = -\lambda(E(\theta_t, \dot{\theta}_t) - E_{\text{up}})^2 - \mu x_t^2$$

모든 변수의 계산 방법을 알았다. 그러면 실제 simulation 환경에서 변수들의 계산 순서는 어떻게 되는가?

Time step calculation - 1. 상태 입력

$$s_t = [x_t, \dot{x}_t, \theta_t, \dot{\theta}_t]$$

타임스텝 t 에서 PPO 정책 네트워크 π_θ 가 Gaussian 정책이라고 가정

Time step calculation - 2. 정책 네트워크 평균, 분산 출력

$$(\mu_\theta(s_t), \sigma_\theta(s_t))$$

타임스텝 t 에서 PPO 정책 네트워크 π_θ 가 Gaussian 정책이라고 가정

Time step calculation - 3. 행동 샘플링

$$a_t \sim \mathcal{N}(\mu_\theta(s_t), \sigma_\theta(s_t)^2)$$

타임스텝 t 에서 PPO 정책 네트워크 π_θ 가 Gaussian 정책이라고 가정

Time step calculation - 4. 물리 시뮬레이션

$\ddot{x}_t, \ddot{\theta}_t$ 계산 후

Δt 적분해 s_{t+1} 계산

타임스텝 t 에서 PPO 정책 네트워크 π_θ 가 Gaussian 정책이라고 가정

Time step calculation - 5. 보상 계산

$$r_t = -\alpha(\theta_t - \pi)^2 - \beta\dot{\theta}_t^2 - \gamma x_t^2 - \delta\dot{x}_t^2$$

타임스텝 t 에서 PPO 정책 네트워크 π_θ 가 Gaussian 정책이라고 가정

Time step calculation - 6. PPO

advantage A_t 계산

파라미터 θ 업데이트

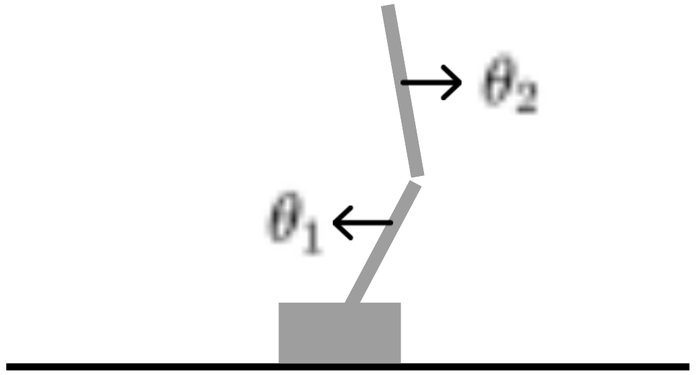
타임스텝 t 에서 PPO 정책 네트워크 π_θ 가 Gaussian 정책이라고 가정

Cart-pole의 계산과정을 알게 되었다. 그러면 막대가 하나 추가된 이중역진자의 변수는 어떻게 달라지는가?

Double Inverted Pendulum Swing-up : state & action

$$s = \begin{bmatrix} x \\ \dot{x} \\ \theta_1 \\ \dot{\theta}_1 \\ \theta_2 \\ \dot{\theta}_2 \end{bmatrix}$$

대표적 상태 벡터



θ_1 : 첫번째 상태 각도

θ_2 : 두번째 상태 각도

기본 state의 instance는 동일하다는 사실을 알게 되었다. 그러면 동역학 방정식은 어떤 차이가 있을까?

동역학 방정식 (개념)

$$q = \begin{bmatrix} x \\ \theta_1 \\ \theta_2 \end{bmatrix}$$

일반화 좌표

$$M(q)\ddot{q} + C(q, \dot{q}) + G(q) = Bu$$

라그랑지안으로부터 얻는 동역학

$M(q)$: 3×3 질량/관성 행렬

$C(q, \dot{q})$: 코리올리/마찰항

$G(q)$: 중력항

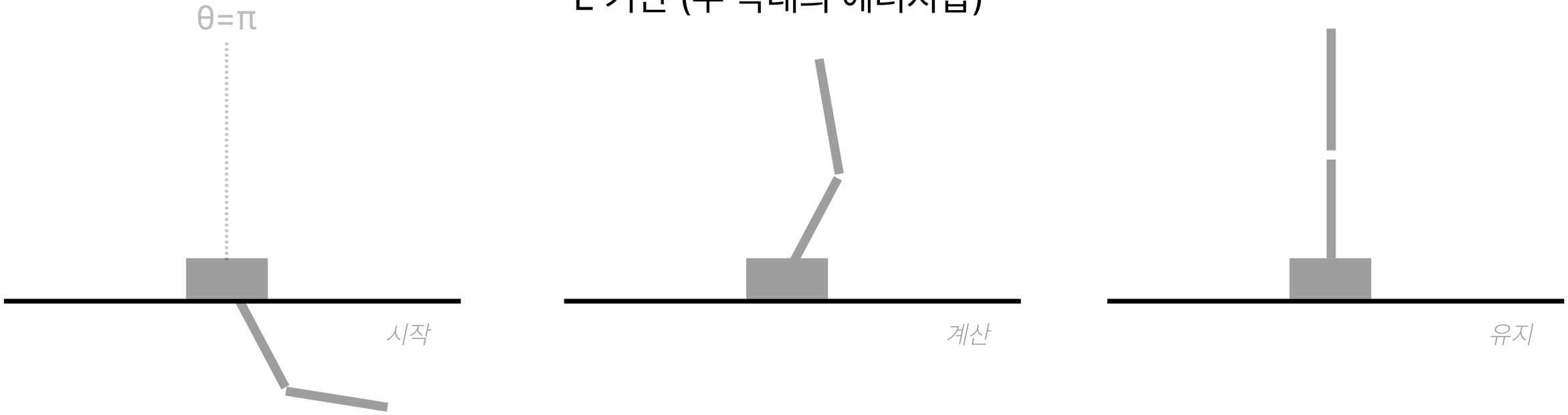
$u = F$: 카트에 가하는 힘 (1차원)

B : 입력 행렬 3×1

Double Inverted Pendulum Swing-up : reward

두 막대 모두 위로 세우는 것 ($\theta_1 \approx \pi$, $\theta_2 \approx \pi$)

E 기반 (두 막대의 에너지합)



$$r_t = -\alpha_1(\theta_{1,t} - \pi)^2 - \alpha_2(\theta_{2,t} - \pi)^2 - \beta_1\dot{\theta}_{1,t}^2 - \beta_2\dot{\theta}_{2,t}^2 - \gamma x_t^2 - \delta\dot{x}_t^2$$

막대가 하나인 Cart-pole에서는 보상을 각도기반, E기반으로 주었는데, 이중역진자에서 다른 방법은 없을까?

Double Inverted Pendulum Swing-up : reward

비선형 상호작용이 더 강하기 때문에, 보상 설계를 잘못하면 학습이 어려움

“둘 다 동시에 위로 가야” 높은 보상이 주어지도록 설계하면 local optimum에 빠지기 쉬움

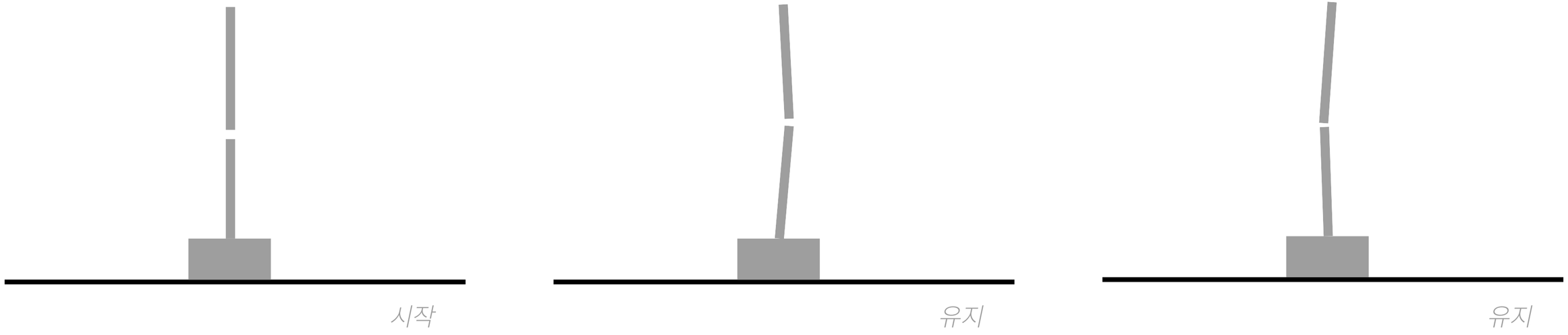
“하나만 세워도 약간의 보상, 둘 다 세우면 큰 보상”처럼 shaped reward 를 쓰는 경우도 많음

보상에 로그함수 사용

Triple-Quintuple Inverted Pendulum Balancing

"스윙업"이 아니라 이미 어느 정도 위로 세워진 상태에서 "밸런싱"을 유지

평형점 근처에서 작은 오차를 유지



Swing-up과 Balancing의 기본 state vector의 instance에는 어떤 차이가 있는가?

State and Action

n 개의 막대를 가진 n-중 역진자

$$s = \begin{bmatrix} x \\ \dot{x} \\ \theta_1 \\ \dot{\theta}_1 \\ \theta_2 \\ \dot{\theta}_2 \\ \cdots \\ \theta_n \\ \dot{\theta}_n \end{bmatrix} \in \mathbb{R}^{2(n+1)}$$

상태

State vector의 instance에는 차이가 없다는 사실을 알게 되었다. 동역학 방정식에는 어떤 차이가 있는가?

동역학 방정식 (개념)

$$q = [x, \theta_1, \ldots, \theta_n]^T \qquad M(q)\ddot{q} + C(q, \dot{q}) + G(q) = Bu$$

일반화 좌표

달라진점 M: $(n+1) \times (n+1)$ 크기 $\quad u \in \mathbb{R}^m$ (입력 개수)

reward는 어떻게 계산하는가?

밸런싱 보상 설계

스윙업이 아니라 “위쪽 평형점을 유지”가 목적

$$q^* = [x^*, \theta_1^*, \ldots, \theta_n^*] = [0, \pi, \ldots, \pi]$$

평형점

$$e_q = q - q^*, \quad e_{\dot{q}} = \dot{q} - 0$$

상태오차

$$r_t = -e_t^T Q e_t - u_t^T R u_t$$

quadratic 보상

밸런싱 보상 설계

$$e_t = \begin{bmatrix} e_q \\ e_{\dot{q}} \end{bmatrix} \qquad Q \succeq 0,\ R \succ 0$$

가중치 행렬

$$r_t = -\sum_{i=1}^{n} \alpha_i (\theta_{i,t} - \pi)^2 - \sum_{i=1}^{n} \beta_i \dot{\theta}_{i,t}^2 - \gamma x_t^2 - \delta \dot{x}_t^2 - \rho u_t^2$$

막대 개수가 늘어날수록 상태 차원이 커지고 평형점 근처의 안정 영역이 좁아짐

PPO등 정책 기반 방법으로 학습 - exploration 노이즈, 정규화, reward scaling에 민감

학습환경이 PPO라고 반복적으로 언급했는데, PPO가 무엇인가?

PPO(Proximal Policy Optimization) - 정책 업데이트

기본 policy gradient는 목적함수 $J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta}\left[\sum_{t=0}^{T} \gamma^t r_t\right]$ 최대화

기존 policy gradient와의 차이는 무엇인가?

PPO(Proximal Policy Optimization) - 정책 업데이트

reinforce, actor-critic

$$\nabla_\theta J(\theta) \approx \mathbb{E}\left[\nabla_\theta \log \pi_\theta(a_t|s_t)\, A_t\right]$$

A_t는 Advantage

PPO만의 특징에는 무엇이 있는가?

PPO - probability ratio

PPO에서는 이전 정책 파라미터 old θ 에서 데이터를 수집하고,

그 데이터를 사용해 새로운 θ 를 업데이트

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

중요도 비율

>1 이면 새로운 정책이 해당 행동을 더 자주 내보내려는 것

<1 이면 덜 자주 내보내려는 것

Probability ratio이외의 다른 특징이 있는가?

PPO - loss clipping

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\, A_t \right) \right]$$

PPO의 정책 목적함수

ϵ은 보통 0.1~0.2

$\mathrm{clip}(r, 1-\epsilon, 1+\epsilon)$: r 이 이 범위를 벗어나면 잘라 버림

또 무슨 특징이 있는가?

PPO - loss clipping

$$L^{\mathrm{CLIP}}(\theta) = \mathbb{E}_t \left[\min \left(r_t(\theta) A_t,\ \mathrm{clip}(r_t(\theta), 1-\epsilon, 1+\epsilon)\, A_t \right) \right]$$

PPO의 정책 목적함수

min 구조

클리핑이 걸렸을 때는 gradient가 더 이상 커지지 않고 제한

policy가 한 번의 update에서 너무 많이 변하는 것을 막음

Value function loss과 entropy bonus는 어떻게 계산되는가?

Value function loss & entropy bonus

actor-critic 구조를 써서, value function도 함께 학습

$$L^{\mathrm{VF}}(\phi) = \mathbb{E}_t \left[(V_\phi(s_t) - V_t^{\mathrm{target}})^2 \right]$$

Value 손실

V_t^{target} : Generalized Advantage Estimatio으로 계산한 반환값

Value function loss & entropy bonus

정책이 너무 deterministic해지는 것을 막기 위해 entropy 보너스 추가

$$L^{\mathrm{ENT}}(\theta) = \mathbb{E}_t\left[\mathcal{H}(\pi_\theta(\cdot|s_t))\right]$$

entropy bonus

$$L^{\mathrm{PPO}}(\theta,\phi) = \mathbb{E}_t\left[L^{\mathrm{CLIP}}(\theta) - c_1 L^{\mathrm{VF}}(\phi) + c_2 L^{\mathrm{ENT}}(\theta)\right]$$

PPO 목적

최대화하는 방향으로 gradient ascent

실제 simulation 상황에서는 어떻게 계산되는가?

each iteration calculation - 1. roll out 수집

1. 현 policy $\pi_{\theta_{\text{old}}}$를 사용하여 여러 episode나 T스텝짜리 trajectory 생성

2. 각 타임스텝에서 (s_t, a_t, r_t, s_{t+1}), $\pi_{\theta_{\text{old}}}(a_t|s_t)$, $V_{\phi_{\text{old}}}(s_t)$ 저장

each iteration calculation - 2. return & advantage 계산

나중부터 거꾸로 돌아가며 계산 (GAE 사용)

$$\delta_t = r_t + \gamma V_{\phi_{\text{old}}}(s_{t+1}) - V_{\phi_{\text{old}}}(s_t)$$

$$A_t = \delta_t + \gamma \lambda A_{t+1}$$

$$V_t^{\text{target}} = A_t + V_{\phi_{\text{old}}}(s_t)$$

모든 타임스텝에 대해 A_t 와 V_t^{target} 구함

each iteration calculation - 3. policy update

배치에서 미니배치들을 반복해서 꺼내며 새로운 $\pi_\theta(a_t|s_t)$ 계산

$$r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{\text{old}}}(a_t|s_t)}$$

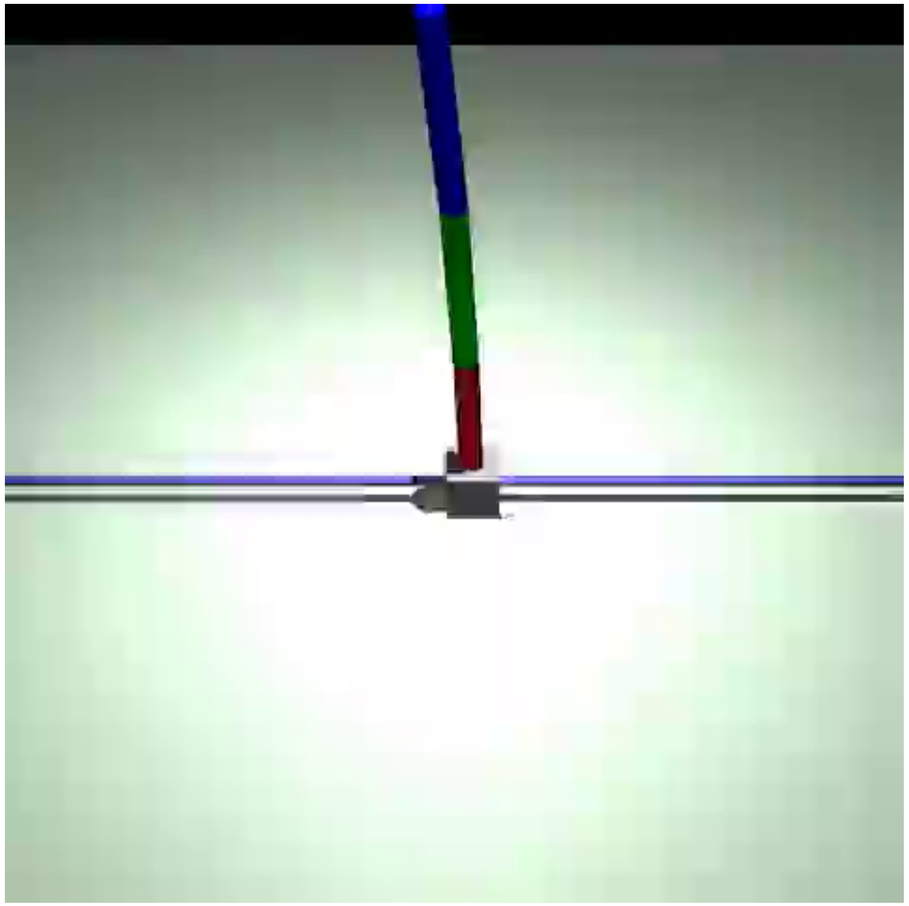
비율

ㅁvalue loss와 entropy까지 포함한 전체 손실 계산후

θ, ϕ 에 대해 backprop 및 optimizer step 수행

$\theta_{\text{old}} \leftarrow \theta$, $\phi_{\text{old}} \leftarrow \phi$ 로 업데이트 후, 다음 iteration 진행

visualization - Double Inverted Pendulum Swing-up



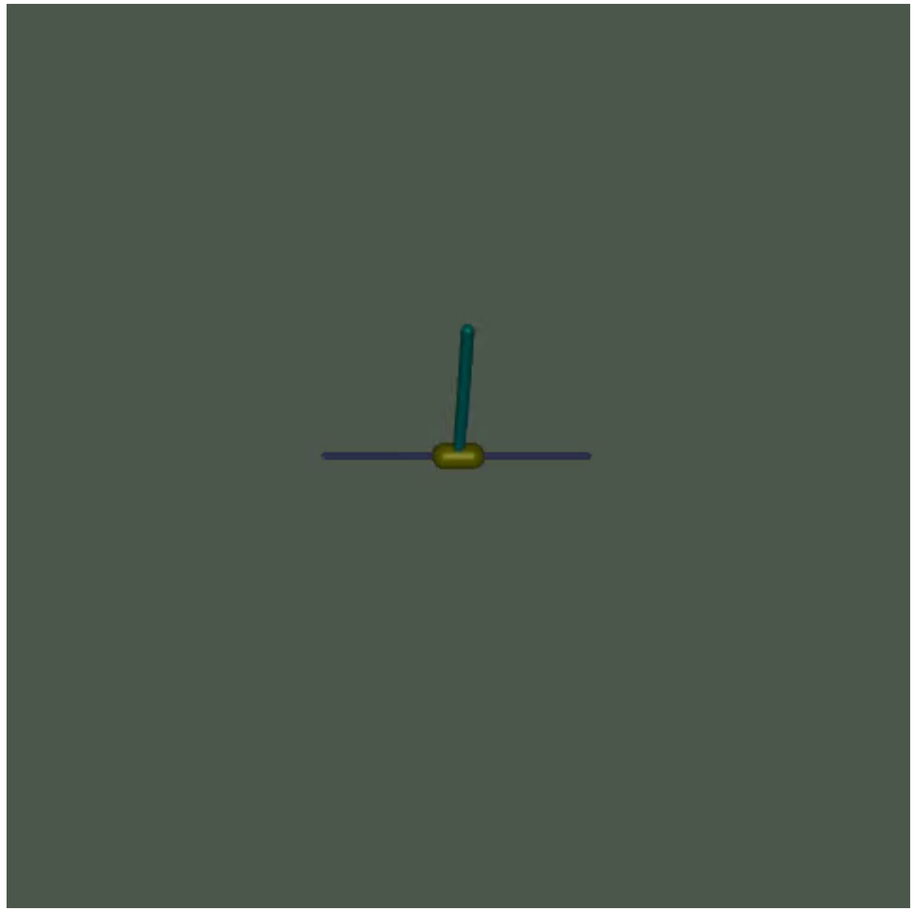
실패이유

보상 설정

보상이 너무 커 정상적인 학습 어려움

보상을 어떻게 설정해야 잘 학습할 수 있을까?

visualization - Double Inverted Pendulum Swing-up



실패이유

보상 설정

보상이 너무 커 정상적인 학습 어려움

학습량의 부족

학습량을 얼마나 설정해야 잘 되는가?

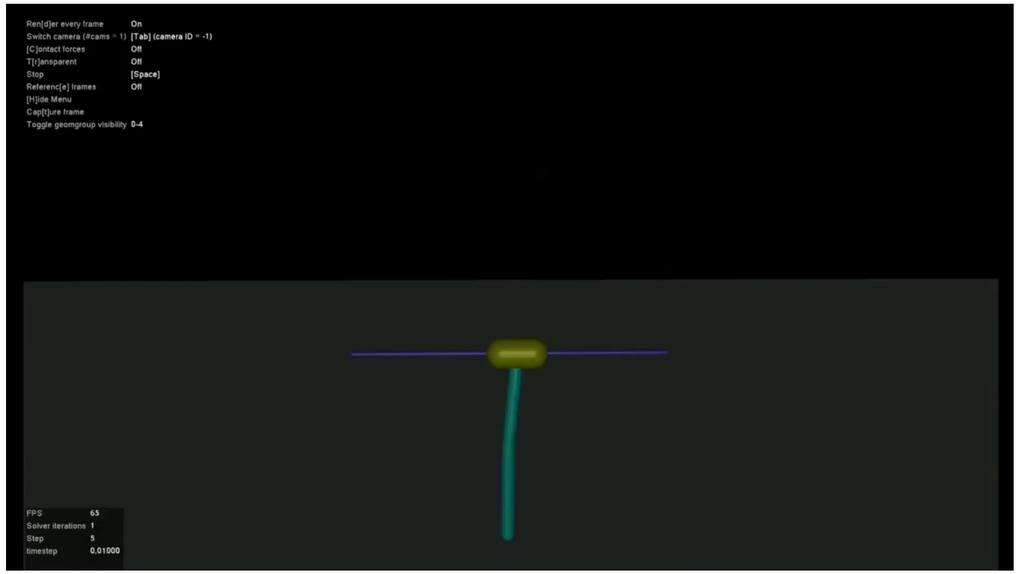
visualization - Double Inverted Pendulum Swing-up

설명

보상을 로그로 설정

충분한 학습량

visualization - Double Inverted Pendulum Swing-up

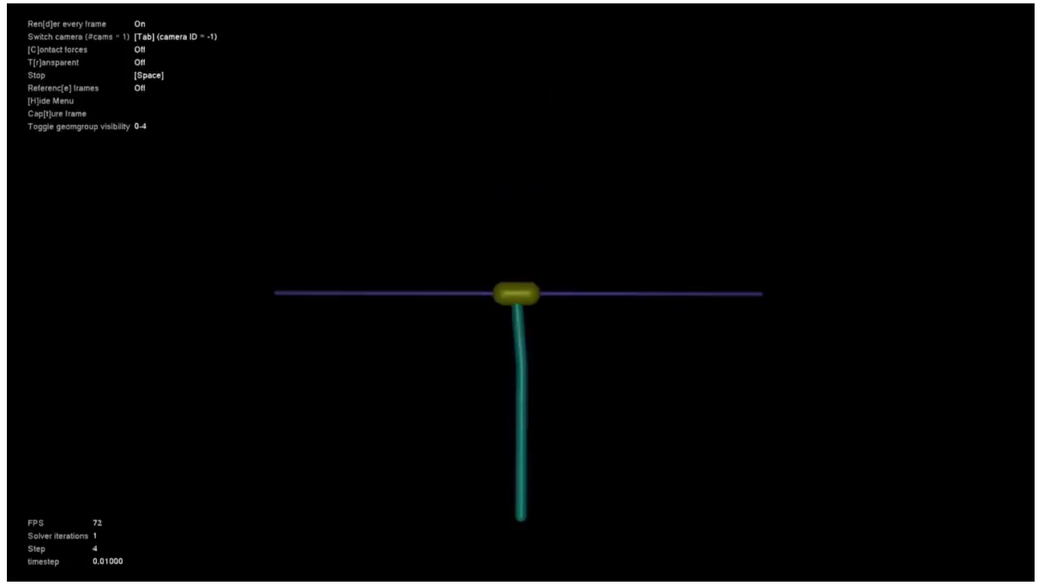


설명

보상을 로그로 설정

충분한 학습량

visualization - Triple Inverted Pendulum Swing-up

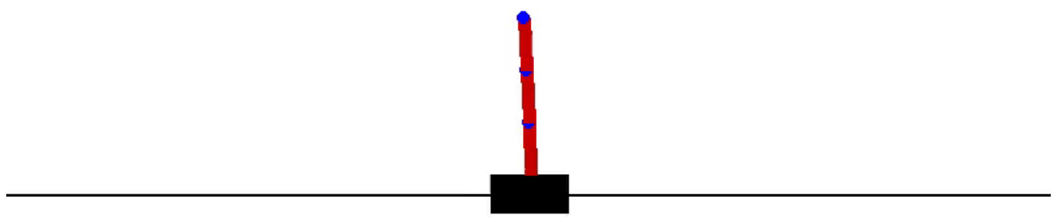


설명

시도중

학습량 부족으로 수요일 이후 완성 예정

visualization - Triple Inverted Pendulum Balancing



설명

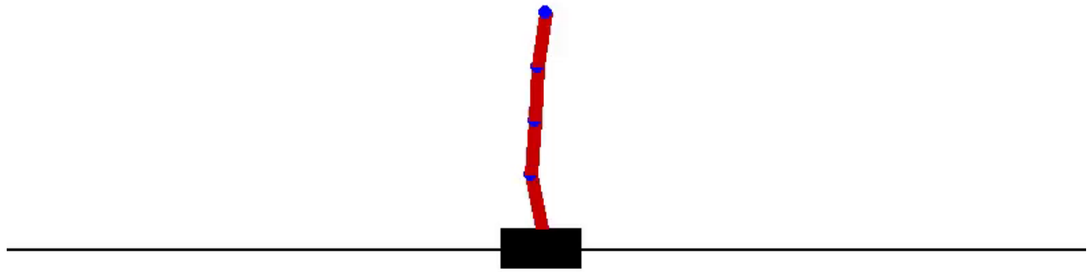
밸런싱

균형이 무너진 것처럼 보이지만,

카트 위 막대는 균형 계산

100초 (limit)동안 유지

visualization - Quadraple Inverted Pendulum Balancing



설명

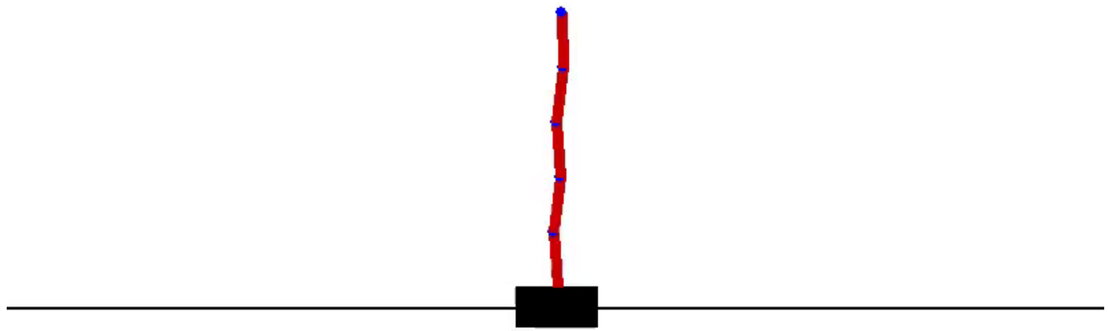
밸런싱

균형이 무너진 것처럼 보이지만,

카트 위 막대는 균형 계산

43초동안 유지

visualization - Quintuple Inverted Pendulum Balancing



설명

밸런싱

균형이 무너진 것처럼 보이지만,

카트 위 막대는 균형 계산

44초동안 유지

Deep-Reinforcement learning Tasks